AI 623 - PA1
Diffusion Models from First Principles

1 Introduction: goals, format, and expectations

Important: this assignment is not graded and has no deliverables. There is **no submission** for this assignment and **no marks** are awarded for producing code, plots, or results. Instead, assessment of the PA content will occur through two separate components:

- a **viva** (oral examination), and
- a **written test** (likely a short quiz).

The purpose of this assignment is therefore *preparatory*: it is a structured way to build the understanding and implementation intuition that you will be asked to demonstrate in the viva and written test. **Please aim to complete this assignment by 1st March 2026.**

Maintain a working document (strongly encouraged). Even though nothing is submitted, you are strongly encouraged to maintain a "working document" (notes + screenshots + plots + short explanations) as you progress.

Required reading before you start. This assignment is designed to be worked through *after* you have read **chapters 2-4** of the course book that cover diffusion / score matching / generative modeling. You are expected to read those chapters **before attempting** the analytical or coding tasks, since the problems assume familiarity with the underlying probability and estimation ideas.

In addition, you must read the **original DDPM paper**. The intent is not to copy implementations from external frameworks, but to connect the paper's definitions and derivations directly to your own code and to the notation used here.

2 Datasets (choose one main + one toy)

- **Toy:** one 2D dataset such as 8-Gaussians, two moons, swiss roll, spiral.
- **Images:** MNIST or FashionMNIST (recommended) *or* CIFAR-10 (harder).

3 Analytical Section

3.1 Notation dictionary

To reduce ambiguity, we will use the following conventions throughout:

- $q(x_0)$ denotes the **data distribution** (this is the same object as p_{data} in some score-matching references).
- $q(x_{1:L} \mid x_0)$ and $q(x_i \mid x_{i-1})$ denote the **forward (noising) process**.
- $p_\theta(x_{0:L})$ and $p_\theta(x_{i-1} \mid x_i)$ denote the **learned reverse model**.
- $p(x_L) = \mathcal{N}(0, I)$ denotes the **prior** at the final diffusion time.

3.2 Problems

Problem 1 (Forward process)**.**

1. **Derive the closed-form marginal.** Starting from (1), prove (2). Your proof should be explicit and identify where Gaussianity is used.
2. **Signal-to-noise ratio.** In this assignment, define

$$\mathrm{SNR}(i) := \frac{\bar{\alpha}_i}{1 - \bar{\alpha}_i},$$

   which corresponds to the signal and noise *power* coefficients in (2). Prove monotonicity under typical schedules (state assumptions) and interpret which timesteps are "hardest" and why.
3. **Analyse** Suppose $\bar{\alpha}_i$ decays too quickly. Which fails first, and why (give a rigorous argument, not a guess)?
   - (A) ELBO tightness in (10),
   - (B) learnability of reverse conditionals,
   - (C) numerical stability of sampling.

   Your answer must reference a specific term in (11) and a specific phenomenon in the forward chain.

Problem 2 (The key Gaussian posterior)**.**

1. **Derive $q(x_{i-1} \mid x_i, x_0)$.** Derive (5)–(7) from (1) and (2). Show the algebra (Gaussian conditioning / completing the square).
2. **Posterior geometry.** Show that $\tilde{\beta}_i < \beta_i$ for $i \geq 2$ under standard schedules and interpret why the posterior variance is smaller than the forward-step variance.
3. **Conceptual check.** Explain in your own words why (5) acts like a "teacher" distribution for training the reverse chain.

Problem 3 (ELBO decomposition and meaning)**.**

1. **Derive the ELBO.** Starting from (10) and (3), derive (11). Make explicit where you use Markov structure and where you use Bayes' rule.

2. **Interpretation as "matching conditionals".** Explain how $\mathcal{L}_{\text{diffusion}}$ forces $p_\theta(x_{i-1} \mid x_i)$ to approximate the family $q(x_{i-1} \mid x_i, x_0)$ for every i.

3. **Analyse** Provide a mathematically precise statement of what it means for $\mathcal{L}_{\text{prior}} = \text{KL}(q(x_L \mid x_0) \| p(x_L))$ to be "small" due to a schedule: choose a divergence or distance, state conditions on $\{\beta_i\}$ that make it small, and justify the choice of measure.

Problem 4 (ϵ-prediction, the simple loss, and what is lost)**.**

1. **Derive the ϵ-parameterized mean.** Starting from (7), derive (8) by expressing an estimator of x_0 in terms of predicted noise.

2. **From ELBO to $\mathcal{L}_{\text{simple}}$.** Explain (with equations) why the diffusion KL terms reduce to a *weighted* squared error in ϵ, and why practitioners often drop weights to get (9).

3. **Analyse** When you drop timestep weights, what are you no longer optimizing faithfully? Tie your argument to (11) and describe one concrete empirical symptom you would expect.

Problem 5 (Tweedie's identity and denoising–score equivalence)**.**

1. **Prove Tweedie's identity** (13)**.** Provide a complete derivation (no skipping the core steps). Hint: differentiate $\log p_\sigma(\tilde{x})$ and use Bayes' rule.

2. **Equivalence.** Use (13) to derive an explicit mapping between an optimal denoiser $\hat{x}_0(\tilde{x})$ and the score $s_\sigma(\tilde{x}) = \nabla_{\tilde{x}} \log p_\sigma(\tilde{x})$.

3. **Analyse** In what precise sense is predicting ϵ an "easier" target than predicting the score directly? Choose two of: variance scaling, Lipschitzness, conditioning on t, gradient magnitude distribution. Argue with math and (optionally) a small toy example.

Problem 6 (Unifying DDPM and score matching)**.**

1. **Map ϵ-prediction to a score estimate.** Using $x_i = \sqrt{\bar{\alpha}_i} x_0 + \sqrt{1 - \bar{\alpha}_i}\epsilon$, show how an ϵ-predictor $\epsilon_\theta(x_i, i)$ induces a score estimate for an appropriate smoothed density at "time" i. State precisely what distribution's score you are estimating.

2. **Effective noise level.** Derive an expression for an "effective" σ_i that puts x_i into a signal-plus-noise form comparable to $\tilde{x} = x_0 + \sigma\epsilon$.

3. **Analyse** Which parameterization is better conditioned in high dimensions: predicting ϵ, predicting x_0, or predicting a linear combination (v-param)? Define what you mean by "better conditioned" and defend with a concrete argument.

4 Coding Section

4.1 Purpose and scope

The coding section is designed to make you *reimplement DDPM end-to-end* and understand every moving part: the schedule, forward noising, posterior formulas, training objective, and sampling procedure.

Implementation Guidance

What you must have working by the end (minimum functional DDPM):

1. A correct implementation of the forward process (1) and the closed-form marginal (2).
2. A correct implementation of posterior quantities $\tilde{\mu}_i, \tilde{\beta}_i$ in (6)–(7).
3. A neural network $\epsilon_\theta(x_i, i)$ with timestep conditioning.
4. Training with $\mathcal{L}_{\text{simple}}$ in (9).
5. Ancestral sampling from $x_L \sim \mathcal{N}(0, I)$ down to x_0 using the reverse chain with variance choice specified in the Appendix.

Everything else (extra samplers, score matching, SDE/ODE) is optional.

4.2 Baseline configuration

Implementation Guidance

You can use the following baseline configuration as reference. You are free to make any modifications.

- **Dataset:** MNIST or FashionMNIST.
- **Data scaling:** map to $[-1, 1]$.
- **Timesteps:** $L = 1000$.
- **Schedule:** linear β_i from $\beta_{\min} = 10^{-4}$ to $\beta_{\max} = 0.02$.
- **Model:** small U-Net, channel multipliers $(32, 64, 128)$, 2–3 resolution levels, 1 residual block per level, GroupNorm + SiLU, no attention.
- **Optimizer:** Adam or AdamW, learning rate 2×10^{-4}, weight decay 0 (or a clearly stated value).
- **Batch size:** as large as fits GPU memory.
- **Training budget:** at least 100k gradient steps (or at least 50 epochs if training by epochs).
- **Sampling:** ancestral sampling with all L steps for the main baseline sample grid.

Minimum baseline artifacts to investigate:

- schedule plots ($\bar{\alpha}_i$ and $\mathrm{SNR}(i)$),
- training loss curve,
- final sample grid (at least 64 samples),
- denoising trajectory visualization for a fixed random seed (save several intermediate x_i).

4.3 Recommended code organization (suggested, not mandatory)

Implementation Guidance

Keep diffusion math separate from the neural net. A coherent minimal structure:

- `diffusion/schedule.py`: compute $\beta_i, \alpha_i, \bar{\alpha}_i$ and any precomputed scalars used everywhere.
- `diffusion/forward.py`: `q_sample`, utilities to draw (i, ϵ, x_i).
- `diffusion/posterior.py`: `q_posterior_mean_var` returning $(\tilde{\mu}_i, \tilde{\beta}_i)$; utilities to compute $\hat{x}_0$ from $\hat{\epsilon}$.
- `diffusion/ddpm.py`: sampling functions (ancestral sampler; optional deterministic sampler if you implement one).
- `models/unet.py`: $\epsilon_\theta(x, i)$ network.
- `train.py`: training loop + logging + periodic sampling.
- `eval.py`: utilities to compute simple diagnostics and generate sample grids.

4.4 Resources (docs and reference material)

Resources (docs, libraries, references)

Documentation you should use (not optional):

- **PyTorch:** `torch.nn`, `torch.optim`, `torch.utils.data`, and mixed precision (`torch.cuda.amp`) if desired.
- **Torchvision:** datasets (`MNIST`, `FashionMNIST`, `CIFAR10`) and transforms.
- **Matplotlib:** save image grids over training and plot schedule diagnostics.

Allowed helper libraries (recommended):

- `einops` (tensor reshaping), `tqdm` (progress bars), `numpy`.

Primary references for DDPM (read + explore; do not copy diffusion utilities):

- **Original DDPM paper (Ho et al., 2020):** arXiv:2006.11239
- **DDPM project page (author):** hojonathanho.github.io/diffusion/.
- **Official code release by the author (reference implementation):** github.com/hojonathanho/diffusion.

Architecture-only references:

- **U-Net (original):** *U-Net: Convolutional Networks for Biomedical Image Segmentation* (Ronneberger et al., 2015).
  arXiv:1505.04597
- **PyTorch building blocks to read about (highly recommended):**
  - `torch.nn.Conv2d`, `torch.nn.GroupNorm`, `torch.nn.SiLU`, `torch.nn.Upsample`, `torch.nn.Dropout`.
  - `torch.nn.Module`, `torch.nn.Sequential`, and `torch.nn.functional`.
- **Implementation guidance:**
  - Start with a **small U-Net** for MNIST/FashionMNIST: 2–3 resolution levels, channel multiplier like $(32, 64, 128)$, and one residual block per level.
  - Use **skip connections** between encoder and decoder at matching resolutions.
  - Inject **timestep embeddings** into residual blocks by:
    - computing an embedding $e(i) \in \mathbb{R}^d$ (sinusoidal or learned),
    - passing it through an MLP to match channel dimension,
    - adding it as a bias term to intermediate activations (e.g. after the first conv in the block).
  - Use **GroupNorm + SiLU** as your default normalization/activation pair.
  - Keep attention **off initially**. Only add (single-head) self-attention at the bottleneck once the baseline works.

4.5 Task 0: Setup and dataset sanity

Objective

Set up the dataset pipeline and verify basic preprocessing. You should do this *before* writing diffusion code.

Implementation Guidance

Requirements:

1. Use MNIST or FashionMNIST (recommended for the assignment). CIFAR-10 is allowed but substantially harder.
2. Define a transform that maps images to a known range, either:
   - $x \in [-1, 1]$ (recommended), or
   - $x \in [0, 1]$ (allowed, but you must be consistent everywhere).
3. Create a dataloader and confirm:
   - tensor dtype and shape (e.g. `(B,1,28,28)`),
   - min/max values are in the intended range.
4. Save a grid of real images as a reference baseline.

Common Pitfalls & Debugging

If you get unstable training or washed-out samples later, the culprit is often inconsistent data scaling (e.g. training in $[-1, 1]$ but sampling interpreted as $[0, 1]$).

4.6 Task 1: Schedules and forward process

Objective

Implement schedules and forward noising (1)–(2). This is the foundation of everything.

Implementation Guidance

1. `make_beta_schedule(L, type, beta_min, beta_max)` returning a tensor `betas` of length L.
2. Precompute and store (as tensors on the correct device):

$$\alpha_i,\ \bar{\alpha}_i,\ \sqrt{\bar{\alpha}_i},\ \sqrt{1-\bar{\alpha}_i},\ 1/\sqrt{\alpha_i}.$$

3. `q_sample(x0, i, eps)` implementing (2).
4. `sample_timesteps(batch_size)` that samples $i \sim \mathrm{Unif}\{1, \ldots, L\}$.

Verification:

1. Plot $\bar{\alpha}_i$ and $\mathrm{SNR}(i) = \bar{\alpha}_i/(1-\bar{\alpha}_i)$ for your schedule.
2. Fix an image x_0, pick a timestep i, sample many x_i and empirically verify the mean and variance implied by (2).

Common Pitfalls & Debugging

Indexing convention: your code must be explicit about whether timesteps are stored as `0..L-1` or `1..L`. Pick one and enforce it everywhere.

4.7 Task 2: Posterior formulas and the reverse-step mechanics

Objective

Implement the true posterior parameters (5)–(7) and the reverse mean (8).

Implementation Guidance

1. `q_posterior_mean_var(x0, xi, i)` returning:

$$\tilde{\mu}_i(x_i, x_0), \ \tilde{\beta}_i.$$

2. `predict_x0_from_eps(xi, i, eps_hat)`:

$$\hat{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_i}} \left(x_i - \sqrt{1 - \bar{\alpha}_i}\, \hat{\epsilon} \right).$$

3. `p_mean_from_eps(xi, i, eps_hat)` implementing (8).

4. `p_sample_step(xi, i, eps_hat)` that returns x_{i-1}:

   - compute μ from `p_mean_from_eps`,
   - if $i > 1$ sample $z \sim \mathcal{N}(0, I)$ and return $\mu + \sqrt{\tilde{\beta}_i}\, z$,
   - if $i = 1$ return μ (no noise).

Verification:

1. Numerically check that $\tilde{\beta}_i > 0$ and (typically) $\tilde{\beta}_i < \beta_i$ for $i \geq 2$.

2. "Teacher consistency" test: generate (x_0, ϵ, x_i) and compute $\tilde{\mu}_i(x_i, x_0)$. Verify it matches the posterior mean formula and points toward plausible x_{i-1} samples.

Common Pitfalls & Debugging

Mixing variance conventions (deriving with $\tilde{\beta}_i$ but sampling with β_i, or vice versa) can be a cause of bad samples.

4.8 Task 3: Build $\epsilon_\theta(x_i, i)$ with timestep conditioning

Objective

Implement a network that takes a noised image x_i and a timestep i, and predicts the noise ϵ used to generate x_i.

Implementation Guidance

Requirements:

1. Input: x_i (image tensor) and timestep i (integer tensor).
2. Output: $\hat{\epsilon}$ with the same shape as x_i.
3. Use a timestep embedding; acceptable options include:
   - sinusoidal embeddings + MLP projection (recommended),
   - learned embedding table (simpler, acceptable for MNIST-scale).
4. Use a U-Net-like architecture (recommended) or a smaller ConvNet (allowed for initial debugging).

Start small. Get a tiny model working and sampling before scaling model size. If CIFAR-10 is chosen, a U-Net is effectively required.

Common Pitfalls & Debugging

If the timestep embedding is unused (e.g. computed but never injected into the network), training loss can still decrease but sampling quality will stagnate.

4.9 Task 4: Train DDPM with $\mathcal{L}_{\text{simple}}$

Objective

Train using (9) and verify correctness with strict sanity checks.

Implementation Guidance

What your training loop must do each iteration:

1. Sample a minibatch x_0.
2. Sample timesteps $i \sim \text{Unif}\{1, \ldots, L\}$.
3. Sample $\epsilon \sim \mathcal{N}(0, I)$.
4. Compute x_i via `q_sample`.
5. Predict $\hat{\epsilon} = \epsilon_\theta(x_i, i)$.
6. Compute loss: mean squared error between ϵ and $\hat{\epsilon}$.
7. Backprop + optimizer step.

Logging you should implement (strongly recommended):

- training loss curve,
- gradient norm or parameter norm (to detect explosions),
- periodic sample grids (even if low quality early).

Sanity checks:

1. **Overfit test:** train on 256 images only. You should see samples resembling the subset.
2. **One-step posterior check:** for a fixed timestep $i \geq 2$, sample (x_0, ϵ, x_i) and compute $\tilde{\mu}_i(x_i, x_0)$. Then draw $x_{i-1} \sim q(x_{i-1} \mid x_i, x_0)$ using $\tilde{\mu}_i$ and $\tilde{\beta}_i$. Empirically verify that $\mathbb{E}[\|x_{i-1} - x_0\|^2] < \mathbb{E}[\|x_i - x_0\|^2]$ over many trials.
3. **Noise-prediction sanity:** for fixed i, compute correlation between ϵ and $\hat{\epsilon}$ across a validation batch.
4. **Timestep sanity:** verify your code samples timesteps uniformly (histogram).

Common Pitfalls & Debugging

If the overfit test or the one-step posterior check fails, do not proceed to ablations/extensions. It indicates a foundational bug in: schedule scalars, forward noising, posterior variance/mean, or timestep conditioning.

4.10 Task 5: Implement the full ancestral DDPM sampler

Objective

Generate samples from the learned reverse chain, using exactly the mechanics implied by the DDPM model.

Implementation Guidance

Required implementation behavior:

1. Start from $x_L \sim \mathcal{N}(0, I)$.
2. For $i = L, L-1, \ldots, 1$:
   - compute $\hat{\epsilon} = \epsilon_\theta(x_i, i)$,
   - compute reverse mean μ via (8),
   - inject noise with variance $\tilde{\beta}_i$ for $i > 1$, none for $i = 1$.
3. Return x_0 and save a grid of samples.

Recommended sampling diagnostics:

- Save intermediate x_i at a few timesteps (e.g. $i \in \{L, 3L/4, L/2, L/4, 1\}$) to visualize denoising progression.
- Track $\|x_i\|$ across timesteps to detect blow-ups.

4.11 Task 6: Focused ablations + one DDPM-adjacent extension

Objective

Perform a *small, coherent* experimental study around DDPM. This replaces the broad earlier tasks.

Implementation Guidance

Possible ablations:

1. **Schedule ablation:** compare linear β_i to one alternative schedule. Record: $\bar{\alpha}_i$ and SNR(i) plots, training curves, and sample grids at matched compute.
2. **Sampling-step ablation:** evaluate the effect of using fewer sampling steps (e.g. by skipping timesteps). Record: sample grids and a failure-mode narrative (what breaks first and why).
3. **Parameterization ablation:** implement x_0-prediction instead of ϵ-prediction. Record: training stability and sample quality; connect to your analytical discussion.

Possible extensions:

- **Deterministic sampling (DDIM-style):** implement a deterministic reverse update that reduces randomness and allows step reduction.
- **Principled timestep weighting:** add timestep-dependent weights to approximate the ELBO-motivated weighting (derive or justify your weight).
- **Schedule design by a formal criterion:** design $\{\beta_i\}$ from a written principle (e.g. equal SNR drop).

Expectations: You should take a hypothesis *before* running the experiments, and then try to explain (with evidence) whether results support it.

Common Pitfalls & Debugging

Ablations are only meaningful if you keep everything else fixed (architecture, optimizer, training steps/epochs, etc.). If you change multiple things at once, you cannot interpret outcomes.

4.12 Task 7: Evaluation and reporting

Objective

Evaluate sample quality *and* diversity with standardized quantitative metrics, plus simple overfitting/memorization checks.

Implementation Guidance

What to produce for yourself:

1. **Qualitative artifacts:**
   - a final sample grid of at least 64 samples (keep the grid size consistent across runs),
   - a fixed-seed denoising trajectory visualization (save x_i at several timesteps, e.g. $i \in \{L, 3L/4, L/2, L/4, 1\}$),
   - (optional) a small "interpolation" grid by interpolating the initial noise x_L between two random seeds.
2. **Quantitative metrics:** Compute at least **two** quantitative metrics appropriate for your dataset (see below).
3. **Likelihood evaluation (optional but informative):** Computer a test-set **NLL / bits-per-dimension** estimate
4. **Overfitting checks:** Run at least **one** memorization/overfit diagnostic (see below), especially if your samples look "too perfect" or if you trained for a long time.

Implementation Guidance

A. Quantitative evaluation for MNIST/FashionMNIST (preferred over ImageNet-FID):
Inception-based FID/KID (ImageNet features) is often *misleading* on MNIST-scale datasets. Instead, compute distances in a feature space learned *on the same dataset*.

1. **Dataset-FID (recommended):**
   - Train a simple classifier on the *real* training set (e.g. a small CNN or LeNet-like model).
   - Extract activations from a fixed intermediate layer as embeddings $\phi(x) \in \mathbb{R}^d$.
   - Compute FID in this embedding space between generated samples and real test samples:

   $$\mathrm{FID}_\phi(\mathcal{X}_{\text{gen}}, \mathcal{X}_{\text{real}}) = \|\mu_g - \mu_r\|_2^2 + \mathrm{tr}\Big(\Sigma_g + \Sigma_r - 2(\Sigma_g\Sigma_r)^{1/2}\Big),$$

   where (μ_g, Σ_g) are the empirical mean/covariance of $\phi(x)$ for generated data and (μ_r, Σ_r) are those for real data.
2. **Dataset-KID (recommended, robust for smaller sample sizes):** Compute KID in the same feature space $\phi(x)$ using a polynomial kernel MMD. (KID is an unbiased estimator and tends to be more stable than FID for limited samples.)
3. **Classifier-based quality + diversity:**
   - Train a classifier on real data (same as above).
   - Record **Accuracy on generated samples** (do generated digits look classifiable?).
   - Record the **entropy** of the predicted class histogram on generated samples: higher entropy indicates broader class coverage; low entropy suggests mode collapse.

Practical notes (MNIST-scale):

- If feasible, use at least 10,000 generated samples for stable estimates; otherwise record your sample count and lean on KID.
- Computing metrics against the **test set** (not the training set) gives a cleaner generalization signal.
- Keep the feature extractor ϕ **fixed** across runs so numbers are comparable.

Implementation Guidance

B. Quantitative evaluation for CIFAR-10 (if you choose it):

- FID (Inception features) and KID are standard and reasonably informative on CIFAR-10.
- (optional) Precision/Recall for generative models in feature space helps separate quality vs diversity.

Implementation Guidance

C. Quantitative evaluation for the 2D toy dataset: Pick at least **one** of the following:

- **Sliced Wasserstein Distance (SWD):** compare generated samples to real samples using random 1D projections.
- **MMD (RBF kernel):** maximum mean discrepancy between generated and real sample sets.
- **Coverage diagnostics:** compare means/covariances per mode (e.g. 8-Gaussians) and record mode coverage.

Also try making a 2D scatter plot overlaying real vs generated samples.

Implementation Guidance

D. Overfitting / memorization checks: If you do only one, do nearest-neighbors.

1. **Nearest-neighbor comparison:** For a batch of generated samples, retrieve and display the nearest training image:

   - nearest in pixel space (ℓ_2 on raw tensors),
   - and/or nearest in feature space (ℓ_2 on $\phi(x)$ embeddings).

   Make a small grid: (generated sample, nearest training neighbor). If generated images closely replicate training images, you are likely memorizing.

2. **Train-vs-test gap:** Compute FID_ϕ or KID against the training set *and* against the test set. A large gap can indicate overfitting or a mismatch in diversity.

3. **Diversity proxy (optional):** Record average pairwise distance in feature space or MS-SSIM (lower similarity $\Rightarrow$ more diversity).

Implementation Guidance

E. Likelihood / NLL:
What is NLL here? For a generative model $p_\theta(x_0)$, the **negative log-likelihood** (NLL) of data is

$$\mathrm{NLL} = \mathbb{E}_{x_0 \sim q}\big[-\log p_\theta(x_0)\big].$$

Lower NLL means the model assigns higher probability to real data. This is useful because many classic baselines (e.g. PixelCNN and variants) report likelihood numbers on common datasets, so you can compare how diffusion models fare under a likelihood metric.
For diffusion models, the exact $-\log p_\theta(x_0)$ is typically *intractable*. What is commonly reported is an **upper bound** on NLL given by the ELBO in (10), so a standard "likelihood-style" report for DDPM is the *average ELBO* on the test set. It is normal that optimizing $\mathcal{L}_{\text{simple}}$ in (9) does *not* perfectly optimize this bound.
How to report it (recommended convention): On image datasets, report **bits-per-dimension** (bpd), defined from an NLL-like quantity by

$$\mathrm{bpd} := \frac{1}{D \log 2}\, \mathbb{E}_{x_0 \sim q}\big[\mathcal{L}_{\text{ELBO}}(x_0; \theta)\big],$$

where D is the number of scalar dimensions (e.g. 28×28 for MNIST, or $32 \times 32 \times 3$ for CIFAR-10). This makes numbers comparable across models.
How to compute the ELBO in practice: Use the decomposition in (11). All KL terms are between Gaussians, so they have closed forms. A practical recipe is:

1. For each x_0 (use the **test set**), sample a forward trajectory $x_{1:L} \sim q(x_{1:L} \mid x_0)$ (or sample only the needed marginals for Monte Carlo estimation).
2. Compute $\mathcal{L}_{\text{prior}} = \mathrm{KL}(q(x_L \mid x_0) \| p(x_L))$ in closed form.
3. For each $i = 2, \ldots, L$, compute the diffusion KL term

$$\mathrm{KL}\big(q(x_{i-1} \mid x_i, x_0) \,\|\, p_\theta(x_{i-1} \mid x_i)\big),$$

   using $\tilde{\mu}_i, \tilde{\beta}_i$ from (6)–(7) and your model mean/variance choice.
4. Handle the reconstruction term $\mathcal{L}_{\text{recon}} = \mathbb{E}_{q(x_1|x_0)}[-\log p_\theta(x_0 \mid x_1)]$ under an explicit observation model (e.g. a Gaussian or discretized-Gaussian likelihood). **Your choice here matters for comparability**, so record it clearly.

Utility and interpretation:

- NLL/bpd captures *probability mass on the data distribution* and is excellent for comparing to likelihood-based baselines.
- NLL/bpd can disagree with perceptual quality: a model can have strong likelihood but mediocre-looking samples, and vice versa. For diffusion, it is best viewed as a complementary diagnostic to FID/KID-style metrics.

Comparability: Likelihood numbers depend on data preprocessing (e.g. dequantization for discrete pixels, scaling range, and the observation model for $p_\theta(x_0 \mid x_1)$). If you compare to PixelCNN numbers, make sure you match (or at least clearly state) the same data conventions used in the baseline.

Common Pitfalls & Debugging

Common evaluation pitfalls:

- Looking only at sample grids: visually pleasing samples can still be low-diversity or memorized.
- Changing the feature extractor ϕ between runs: you lose comparability.
- Computing metrics against the training set only: this can hide memorization.
- Using ImageNet-Inception FID on MNIST without noting the domain mismatch: interpret with caution.
- Reporting NLL/bpd without stating preprocessing / likelihood convention: comparisons become meaningless.

4.13 Debugging checklist (DDPM-specific)

Common Pitfalls & Debugging

When DDPM fails, debug in this order:

1. **Schedule scalars:** print min/max of $\beta_i, \alpha_i, \bar{\alpha}_i, (1-\bar{\alpha}_i)$; plot $\bar{\alpha}_i$ and $\mathrm{SNR}(i)$.
2. **Forward correctness:** empirical mean/variance of x_i matches (2).
3. **Posterior correctness:** $\tilde{\beta}_i$ positive and sensible; using $\tilde{\beta}_i$ (and not β_i) in sampling noise.
4. **Timestep conditioning:** verify the network actually receives timestep information (not just computed and discarded).
5. **Overfit and posterior check:** if it cannot overfit 256 images or fails the one-step posterior check, sampling will not work.
6. **Sampling mechanics:** ensure "no noise at $i = 1$" and that x_L is sampled from $\mathcal{N}(0, I)$.

5 Appendix

5.1 Background: discrete diffusion (DDPM)

5.1.1 Forward (noising) process

Let $x_0 \sim q(x_0)$ denote a data point (e.g. an image). Choose $L \in \mathbb{N}$ diffusion steps and a variance schedule $\{\beta_i\}_{i=1}^{L}$ with $\beta_i \in (0,1)$. Define $\alpha_i := 1 - \beta_i$ and cumulative product $\bar{\alpha}_i := \prod_{j=1}^{i} \alpha_j$, with $\bar{\alpha}_0 := 1$.

The forward Markov chain is:

$$q(x_{1:L} \mid x_0) = \prod_{i=1}^{L} q(x_i \mid x_{i-1}), \qquad q(x_i \mid x_{i-1}) := \mathcal{N}\big(\sqrt{\alpha_i}\, x_{i-1},\, (1-\alpha_i)I\big). \tag{1}$$

Closed-form marginal. For any i,

$$q(x_i \mid x_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_i}\, x_0,\, (1-\bar{\alpha}_i)I\big), \quad \text{equivalently} \quad x_i = \sqrt{\bar{\alpha}_i}\, x_0 + \sqrt{1-\bar{\alpha}_i}\, \epsilon, \;\; \epsilon \sim \mathcal{N}(0, I). \tag{2}$$

With this parameterization, we define the signal-to-noise ratio as

$$\mathrm{SNR}(i) = \frac{\bar{\alpha}_i}{1-\bar{\alpha}_i},$$

corresponding to the signal and noise *power* coefficients in (2).

5.1.2 Reverse model and the true posterior

We learn a reverse-time Markov chain

$$p_\theta(x_{0:L}) = p(x_L) \prod_{i=1}^{L} p_\theta(x_{i-1} \mid x_i), \qquad p(x_L) := \mathcal{N}(0, I). \tag{3}$$

Typically $p_\theta(x_{i-1} \mid x_i)$ is Gaussian:

$$p_\theta(x_{i-1} \mid x_i) = \mathcal{N}\big(\mu_\theta(x_i, i), \Sigma_\theta(i)\big). \tag{4}$$

Variance convention (this assignment). Unless stated otherwise, we fix

$$\Sigma_\theta(i) = \tilde{\beta}_i\, I,$$

i.e. we use the (scalar) posterior variance in sampling.

A crucial analytic object is the **true** forward posterior:

$$q(x_{i-1} \mid x_i, x_0) = \mathcal{N}\big(\tilde{\mu}_i(x_i, x_0), \tilde{\beta}_i I\big), \tag{5}$$

with

$$\tilde{\beta}_i := \frac{1-\bar{\alpha}_{i-1}}{1-\bar{\alpha}_i}\, \beta_i, \tag{6}$$

$$\tilde{\mu}_i(x_i, x_0) := \frac{\sqrt{\bar{\alpha}_{i-1}}\beta_i}{1-\bar{\alpha}_i}\, x_0 + \frac{\sqrt{\alpha_i}(1-\bar{\alpha}_{i-1})}{1-\bar{\alpha}_i}\, x_i. \tag{7}$$

5.1.3 ϵ-parameterization and the "simple" training loss

Define a neural net $\epsilon_\theta(x_i, i)$ and set

$$\mu_\theta(x_i, i) := \frac{1}{\sqrt{\alpha_i}} \left(x_i - \frac{\beta_i}{\sqrt{1-\bar{\alpha}_i}} \, \epsilon_\theta(x_i, i) \right). \tag{8}$$

A widely used objective is the unweighted noise prediction loss:

$$\mathcal{L}_{\text{simple}}(\theta) := \mathbb{E}_{x_0 \sim q,\, i \sim \text{Unif}\{1,\ldots,L\},\, \epsilon \sim \mathcal{N}(0,I)} \left[\|\epsilon - \epsilon_\theta(x_i, i)\|^2 \right], \quad x_i \text{ from (2)}. \tag{9}$$

5.1.4 ELBO decomposition (what DDPM is optimizing)

The negative log-likelihood admits an ELBO, using the forward chain $q(x_{1:L} \mid x_0)$ as the variational distribution:

$$-\log p_\theta(x_0) \le \mathcal{L}_{\text{ELBO}}(x_0; \theta) := \mathbb{E}_{q(x_{1:L}|x_0)} \left[\log \frac{q(x_{1:L} \mid x_0)}{p_\theta(x_{0:L})} \right]. \tag{10}$$

This ELBO decomposes into:

$$\begin{aligned} \mathcal{L}_{\text{ELBO}}(x_0; \theta) = {} & \underbrace{\text{KL}\big(q(x_L \mid x_0) \,\|\, p(x_L)\big)}_{\mathcal{L}_{\text{prior}}} + \underbrace{\mathbb{E}_{q(x_1|x_0)} \big[-\log p_\theta(x_0 \mid x_1) \big]}_{\mathcal{L}_{\text{recon}}} \\ & + \underbrace{\sum_{i=2}^{L} \mathbb{E}_{q(x_i|x_0)} \left[\text{KL}\big(q(x_{i-1} \mid x_i, x_0) \,\|\, p_\theta(x_{i-1} \mid x_i)\big) \right]}_{\mathcal{L}_{\text{diffusion}}}. \end{aligned} \tag{11}$$

Reconstruction term convention (this assignment). In the implementation, the $i = 1$ step is handled by the same ϵ-parameterized mean in (8) with the convention that no additional noise is injected at $i = 1$ during sampling.

5.2 Background: score matching and SDE/ODE views

5.2.1 Score and denoising score matching (DSM)

For a density $p(x)$, the **score** is $\nabla_x \log p(x)$. Define a Gaussian-smoothed density (note: here p_{data} is the same object as $q(x_0)$):

$$p_\sigma(x) := \int p_{\text{data}}(x_0) \, \mathcal{N}(x \mid x_0, \sigma^2 I) \, \mathrm{d}x_0. \tag{12}$$

Let $\tilde{x} = x_0 + \sigma\epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$.

Tweedie's identity (key link).

$$\mathbb{E}[x_0 \mid \tilde{x}] = \tilde{x} + \sigma^2 \nabla_{\tilde{x}} \log p_\sigma(\tilde{x}). \tag{13}$$

Noise-conditional score model. We learn $s_\theta(x, \sigma) \approx \nabla_x \log p_\sigma(x)$ by minimizing:

$$\mathcal{L}_{\text{DSM}}(\theta; \sigma) := \mathbb{E}_{x_0 \sim p_{\text{data}},\, \epsilon \sim \mathcal{N}(0,I)} \left[\left\| s_\theta(x_0 + \sigma\epsilon, \sigma) + \frac{\epsilon}{\sigma} \right\|^2 \right]. \tag{14}$$

Train across $\{\sigma_\ell\}_{\ell=1}^{M}$ with weights λ_ℓ:

$$\mathcal{L}_{\text{NCSN}}(\theta) := \sum_{\ell=1}^{M} \lambda_\ell \, \mathcal{L}_{\text{DSM}}(\theta; \sigma_\ell). \tag{15}$$

What score appears in DDPM/score equivalence? In the diffusion setting, the score is typically the *marginal* score of the noised variable,

$$q_i(x) := \int q(x_i = x \mid x_0)\, q(x_0)\, \mathrm{d}x_0, \qquad \text{so the relevant score is } \nabla_x \log q_i(x),$$

not the conditional score given a particular x_0.

5.2.2 Annealed Langevin sampling (score-based generation)

Given a trained $s_\theta(x, \sigma)$ and noise ladder $\sigma_1 > \cdots > \sigma_M$, annealed Langevin dynamics is:

$$x^{k+1} = x^k + \eta_\ell\, s_\theta(x^k, \sigma_\ell) + \sqrt{2\eta_\ell}\, z^k, \qquad z^k \sim \mathcal{N}(0, I). \tag{16}$$

run for K steps per noise level (total $O(MK)$ network evaluations).

5.2.3 SDE viewpoint: reverse-time SDE and probability-flow ODE

Consider a forward SDE (here we restrict to isotropic diffusion $g(t)I$):

$$\mathrm{d}x = f(x, t)\, \mathrm{d}t + g(t)\, \mathrm{d}w_t, \qquad t \in [0, T], \tag{17}$$

with density p_t.

Reverse-time SDE. Under standard regularity assumptions, the reverse-time dynamics are:

$$\mathrm{d}x = \big(f(x, t) - g(t)^2 \nabla_x \log p_t(x)\big)\, \mathrm{d}t + g(t)\, \mathrm{d}\bar{w}_t, \tag{18}$$

and sampling is performed by integrating (18) from $t = T$ down to $t = 0$ (i.e. on a reversed time grid).

Probability-flow ODE (PF-ODE). A deterministic ODE with the same marginals:

$$\frac{\mathrm{d}x}{\mathrm{d}t} = f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x). \tag{19}$$

5.2.4 Two canonical SDEs: VE and VP

- **VE (variance exploding):** $\mathrm{d}x = g(t)\, \mathrm{d}w_t$ (zero drift).
- **VP (variance preserving):** $\mathrm{d}x = -\frac{1}{2}\beta(t) x\, \mathrm{d}t + \sqrt{\beta(t)}\, \mathrm{d}w_t$.